Open-Source Harness Gives AI Agents Persistent Identity, Personality, and Team Structure

Punt Labs releases ethos, an agent harness where humans and AI agents work together like a remote-first team, with persistent identity, structured delegation, and composable tool integrations

Press Release

SAN FRANCISCO, July 1, 2026. Punt Labs today announced ethos, a free, open-source agent harness that makes humans and AI agents work together like a remote-first team. Agents get persistent identity (name, personality, writing style, domain expertise) that loads automatically every session and survives context compaction. Teams get structured delegation with typed mission contracts, file-level ownership, and bounded review cycles. Every tool in the developer's stack (messaging, voice, email, semantic search, product evaluation) gains richer context when ethos is present, and works without it when it is not (see FAQ 6).

The insight behind ethos is that agents work best when animated as specialists in a human team pattern. A Go specialist with Kernighan's principles writes better Go than an anonymous agent, not because of magic, but because personality constrains and focuses the model's output the same way a real colleague's expertise would. Models were trained on human collaboration; ethos gives that collaboration structure and persistence. The central hypothesis: do developers want a shared identity and workflow layer for their human-agent team, rather than anonymous agents with prose delegation? (See FAQ 10.)

Problem

Today, when a developer opens an agentic coding tool, the session starts anonymous in both directions. The agent has no personality, no communication style, no team awareness. Sub-agents are interchangeable blanks. Every session begins from scratch [1, 2].

This identity gap is only the first problem. The second is delegation chaos. When a leader assigns work to an agent, the contract is free-form prose: no file ownership, no round budget, no structured result, no audit trail. Two agents can edit the same files simultaneously. A reviewer can silently drift from the criteria it was pinned to at launch. When something goes wrong, the leader reconstructs events from chat history. An audit of 30 popular agent projects found 93% use unscoped credentials with no scope narrowing, depth limits, or delegation chain controls [3]. The industry has agents that can write code but no discipline for assigning, bounding, and verifying the work they do (see FAQ 10).

The third problem is tool fragmentation. A messaging tool reinvents usernames. A voice tool reinvents voice config. An email tool reinvents contact addresses. Each one solves a fragment of identity independently because no shared layer exists. SoulSpec [4] addresses agent personas with structured markdown files, but only for agents, only for personality, and only where the host tool natively loads the files. A2A [5] provides cryptographic Agent Cards for discovery, but has no human identity model and no session persistence.

Solution

Ethos is an agent harness: the shared layer that other tools compose with. It solves three problems with one design principle: agents are teammates, not computation endpoints.

Identity: a developer runs `ethos setup` to create a working team in under 60 seconds. The wizard creates human and agent identities, activates a team bundle where the human is CEO and `claude` is COO over specialist agents (architect, implementer, reviewer, security) that report to the COO, and generates agent definition files. Hooks load identity automatically at session start, survive context compaction, and inject matched personas when sub-agents spawn. Roles define tool restrictions and responsibilities; the team graph derives anti-responsibilities from `reports_to` edges (Feature 3).

Workflow: the leader writes a typed mission contract (worker, write-set, success criteria, evaluator, round budget) and delegates. Pipeline templates chain missions into multi-stage workflows (design → implement → test → review → document). The store refuses overlapping write-sets, pins the evaluator's content hash at launch, and blocks close without a structured result. The leader controls the agency level through contract specificity, not a mode switch (Feature 5).

Composable integrations: every tool in the stack gains richer context when ethos is present and works without it when it is not. This is the core architectural pattern, not a feature but the design thesis:

Tool	Without ethos	With ethos
Biff (messaging)	Team chat	+ identity, presence, team graph
Vox (voice)	Text-to-speech	+ voice persona per agent
Beadle (email)	Send/receive	+ email identity binding
Quarry (search)	Semantic search	+ per-agent mission memory
PR/FAQ	Working Backwards	+ pipeline stage invocation

Any tool reads identity through the pattern that fits: YAML files at a known path (zero dependency), CLI commands from hooks, or the MCP server for structured operations. Extensions let tools attach per-persona config (voice profile, GPG key, routing preferences) without schema changes (Feature 16).

Customer Quote

> *"I run a team of three engineers plus five agent personas: two Go specialists, a security reviewer, an architect, and a CLI designer. Before ethos, delegating to a sub-agent meant writing a paragraph of prose and hoping it stayed on task. Now I write a mission contract: which files to touch, what success looks like, and a cap of two rounds. The agent delivers a structured report that tells me what passed, what didn't, and where to look. The reviewer can't drift between rounds: if someone changes its criteria, the system catches it. A real bug fix went from task assignment to merged PR in under 13 minutes. I didn't make the delegation faster; the contract made it reliable."*
>
> — **Priya Chakraborty**, Staff Engineer at a Series B startup

Getting Started

Two commands, under 60 seconds:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/ethos/7759dd5/install.sh | sh
ethos setup
```

The installer places the binary, seeds starter content (roles, talents, pipelines, two team bundles), and registers the Claude Code plugin (pass `-no-plugin` for a CLI-only install on non-Claude harnesses or plugin-restricted environments). `ethos setup` is an interactive wizard that asks a few

questions (name, handle, email, GitHub handle, working style), then creates human and agent identities, writes the repo config, activates the **foundation** team bundle (a CEO/COO leadership pair over 4 specialist agents: the human is CEO, `claude` is COO, and specialists report to the COO), and generates agent definition files. Start Claude Code. The agent knows who it is, who you are, and how to delegate.

On day one, the team works with defaults. On day three, the developer runs their first pipeline (`ethos mission pipeline instantiate standard`). On day seven, they customize: override a personality, add a domain talent, create a custom pipeline for their review process. The progression is use the defaults, then make them yours.

Spokesperson Quote

> "The hardest design choice was making write-sets a convention, not an enforcement boundary. Agent runtimes don't support filesystem sandboxing today; if we enforced write-sets at the kernel level, we'd be building a sandbox product, not an identity product. So write-sets are declared in the contract, surfaced to the agent, and verified in review. That sounds weak until you see the alternative: every other tool gives agents unscoped access and hopes for the best. We also chose the same schema for humans and agents. That's not idealism; it's plumbing. When the voice tool reads a persona, it doesn't check whether the persona is human or machine. When a mission contract names an evaluator, the system pins the evaluator's content hash regardless of kind. One schema means every integration works for both, and we never maintain parallel paths."
>
> — **Jim Freeman**, Founder at Punt Labs

Call to Action

Ethos is available now at `https://github.com/punt-labs/ethos` under the MIT license. It ships as a single Go binary with zero runtime dependencies. The Claude Code plugin is in the Punt Labs plugin marketplace. A live example walkthrough (real mission, real outputs, real timing) is at `docs/example/` in the repository.

Frequently Asked Questions

External FAQs

Q1: What is ethos and who is it for?

Ethos is an open-source agent harness for engineering teams where humans and AI agents work together like a remote-first team. It provides persistent identity (personality, writing style, domain expertise, team structure) that loads automatically every session; typed mission contracts for structured delegation with file-level ownership, bounded rounds, and audit trails; and a composable integration pattern where every tool in the stack (messaging, voice, email, search, product evaluation) gains richer context when ethos is present and works without it when it is not. Agents are animated as specialists with names, expertise, and communication styles, not anonymous computation endpoints. It is for developers who delegate real work to AI agents and need that delegation to be reliable, personality-consistent, and bounded.

Q2: How is ethos different from SoulSpec, CLAUDE.md, or agent orchestration frameworks?

SoulSpec [4] provides structured agent personas (SOUL.md, IDENTITY.md, STYLE.md) adopted by OpenClaw (370,000+ GitHub stars) [6]. It addresses agent-only personality with no human identity, no team model, no delegation contracts, and no cross-tool integration surface beyond file discovery. CLAUDE.md describes project conventions (coding standards, architecture decisions), not people [7]. Orchestration frameworks like CrewAI and Mastra [8] coordinate agents with roles and typed schemas but have no persistent identity across sessions, no file-level write-set boundaries, and no frozen evaluator to prevent criteria drift.

Ethos is the only tool that combines human-agent parity (same schema for both), typed delegation contracts (write-set, evaluator pinning, bounded rounds), and multi-surface integration (filesystem + CLI + MCP). The bet is that these three capabilities need to be one layer, not three separate tools.

Q3: How do I get started?

Install (`curl -fsSL .../install.sh | sh`), then run `ethos setup` in your project directory. The wizard asks a few questions and creates a working team in under 60 seconds. No account, no cloud, no API key. For non-interactive environments, use `ethos setup -file config.yaml`. On non-Claude harnesses or plugin-restricted environments, install the CLI only with `sh -s -- -no-plugin` (or `ETHOS_NO_PLUGIN=1`).

Q4: How much does it cost?

Free and open-source under the MIT license. Ethos is infrastructure, designed for frictionless adoption, not direct monetization.

Q5: What happens to my data?

Ethos itself makes no network calls and has no server, no telemetry, and no cloud sync. Ethos's own storage leaves your machine only through git, which you control. Context that ethos supplies to an agent (for example the persona and team context injected at session start) is subject to the data policies of the harness and model provider you run. Where data is stored depends on what it is:

- **In the repository, traveling with git.** The identity graph content that is repo-local or vendored into `.punt-labs/ethos/` (identities, personalities, writing styles, talents, roles, teams), and the mission blame record: each mission's contract, structured result, and delegation records, plus the `missions.jsonl` trace log. The identity graph is not repo-local by default: `ethos setup` saves identities under `~/.punt-labs/ethos/`, and resolution reads repo-local, then the active bundle, then global content. Global content and global bundles do not travel with git, so a fresh clone has the complete graph only when the repository vendors it (`ethos vendor`). Identity files are plain YAML, editable with any text editor.
- **On your machine only.** Session state lives in `~/.punt-labs/ethos/sessions`. The live audit and mission files live under `.punt-labs/local/`. The trace tier (delegation prompts, reflections, mission event logs, and per-tool-call audit records) will stay on the machine unless you opt in (see FAQ 18). That is the DES-079 target. Today the tracked mission tree holds delegation prompts (`prompt.md`), `reflections.yaml`, and the sealed event-log chunks. The sealed audit chunks sit in the sessions tree (`.punt-labs/ethos/sessions/<date>-<sid>/`). All of them are committed, as are the agent-written `artifacts/*.md` files (agents write those with their own file tools, and no ethos code does). Under the default `record: blame` the prompts, reflections, and sealed chunks will go to a machine-local archive, `.punt-labs/local/ethos/trace/`. The live event log and live audit file are always machine-local.
- **In your home directory.** `~/.punt-labs/ethos/` holds global starter content (roles, talents, pipelines, team bundles), global identities, session state, and ADRs (`~/.punt-labs/ethos/adrs`). Today ADRs are global and untracked. Storing them in the repository, as part of the blame tier, is decided but not shipped (DES-079, in progress, bead ethos-yjh0).

In progress. This split is decided (DES-079) but not yet implemented: today delegation prompts, reflections, and the sealed event-log chunks are committed in the tracked mission tree, the sealed audit chunks are committed in the sessions tree, and so are the agent-written `artifacts/*.md` files. Agents write those with their own file tools, so they are not an ethos writer and are not in the redaction scope. In both `record` modes, `ethos enable` will write an ignore rule for `.punt-labs/ethos/missions/*/artifacts/`, and ethos never writes, moves, exports, or redacts them. A person who wants an artifact in git moves it elsewhere and commits it, and ethos makes no redaction claim about it. `ethos doctor` lists artifacts that are already tracked, with the `git rm -cached` command. The split also puts ADRs in the repository (as `.punt-labs/ethos/adrs/<id>.yaml`) with the blame tier; today they are global and untracked. Also decided and in progress (DES-080): the mission-record and audit writers will redact absolute paths, email addresses, and credentials. In scope are the mission files, delegation files, the event log, `missions.jsonl`, audit files, the audit sentinel, the legacy-audit migration, `ethos mission migrate`, mission export, and ADRs once they are in the repository. Identity files are not redacted, because they keep their email bindings on purpose. Every in-scope writer will refuse the write when it cannot redact, except the audit writer, which never blocks a tool call. Today the audit writer redacts paths and addresses, and the delegation skeleton writer redacts paths. The event details of three events (abandon, disclaim, write-set force-release) redact paths, and closing a delegation redacts the home directory only. The writers for `contract.yaml`, `results.yaml`, `reflections.yaml`, `missions.jsonl`, the audit sentinel, the legacy-audit migration, `ethos mission migrate`, and mission export do not redact. The rest of the mission event log is unredacted, and no credential pass exists. Already-sealed chunks are not rewritten, because sealed chunks are immutable (DES-058). The mutable records (contract, results, reflections, delegation records and prompts, and `missions.jsonl`) will be scrubbed once, forward only, and past commits are

not rewritten. Redaction is pattern-based and is a backstop. Contracts and commands should not contain credentials.

The repository record has the privacy of the repository. Machine-local data has the privacy of `~/.gitconfig`. 90% of organizations see local storage as inherently safer, and 64% worry about sharing sensitive data with cloud GenAI tools [9].

Q6: What tools integrate with ethos today?

Three Punt Labs tools integrate today, each following the same pattern: **works without ethos, better with it.** Biff (team messaging) gains identity and presence. Vox (voice) gains per-agent voice personas. Beadle (email) gains email identity bindings.

Quarry (semantic search) already supports per-agent memory: the `remember` and `find` tools accept an `agent_handle` parameter, so agents write to and search their own collections. What remains is the ethos-side hooks: mission close prompting the agent for takeaways, and mission dispatch searching the agent's collection for relevant prior work (ethos-d17). The PR/FAQ plugin already produces LaTeX output, runs meeting personas as evaluation gates, and has peer review. The product pipeline's prfaq stage invokes this workflow today.

Three integration surfaces, any coupling level:

- **Filesystem.** Read YAML at known paths. Zero dependency.
- **CLI.** Call `ethos whoami`, `ethos mission show`, `ethos session` from hooks and scripts.
- **MCP server.** 12 tools covering identity, sessions, teams, roles, missions, ADRs, extensions, and vendoring.

The composable integration pattern is the core architectural thesis. Ethos is the harness: it doesn't do the messaging, the voice, the email, or the search. It gives each tool the identity context that makes its output feel like working with a real colleague. The integration surface is open to any third-party tool.

Q7: Can I use missions for design work, not just implementation?

Yes. The mission contract is a spectrum. An implementation mission has a tight write-set ("only this file") and specific criteria ("one ReadFile, make check passes"). A design mission has a broad write-set ("docs/design/") and open-ended criteria ("evaluate three approaches, recommend one with tradeoffs"). The agent's expertise and judgment are always present; the contract determines how much latitude they have. The leader chooses the structure level when writing the contract, not when defining the persona.

Q8: Can I use ethos with SoulSpec or existing CLAUDE.md files?

Yes. Ethos both ingests and exports these formats.

Ingestion. Ethos reads SoulSpec files (SOUL.md, IDENTITY.md, STYLE.md, AGENTS.md) and CLAUDE.md as input during identity creation. The content becomes an ethos identity with structural metadata the source format cannot express: typed roles with tool restrictions, team collaboration graphs, mission contracts with write-set boundaries, safety constraints, and hash-pinned evaluators. The ingested identity is a hardened version of the same content: the personality is the same words, but now it is pinned by content hash so it cannot drift between mission rounds. The writing style is the same prose, but now it is bound to a role that restricts which tools the agent can use.

Export. Ethos exports to SoulSpec format (SOUL.md, IDENTITY.md, STYLE.md, AGENTS.md) for any compatible tool, and to CLAUDE.md for any Claude Code session without ethos installed. Both exports are lossy: structural enforcement (roles, teams, missions, audit trails,

frozen evaluators) drops because markdown cannot represent it. The loss is the point: it shows what ethos adds. Export is useful for distribution (SoulSpec files work in OpenClaw) and portability (CLAUDE.md works without ethos).

The relationship. SoulSpec and CLAUDE.md are content formats: personality as prose, conventions as text. Ethos is the structural layer that makes that content enforceable. Import gains structure. Export loses structure but gains reach.

Internal FAQs

Value & Market

Q9: What is the total addressable market?

Bottoms-up estimate. The immediate market is developers using terminal-based agentic coding tools: Claude Code, Codex CLI, OpenCode. Claude Code is the most-loved AI coding tool at 46% (vs. Cursor 19%, GitHub Copilot 9%) per the Pragmatic Engineer survey ($n \approx 906$) [10]. 71% of agentic-tool users in the Pragmatic Engineer survey use Claude Code. Anthropic's Claude Code run-rate was $2.5B as of February 2026 [11]. We estimate 200,000–500,000 developers in the terminal-first agentic segment based on Claude Code's 115,000 floor (July 2025) [12] plus 2026 growth trajectory.

Assumption: Terminal-first segment size is inferred, not measured. The estimate assumes it grows faster than the broader market because multi-agent workflows are exclusively terminal-based today. Ethos is free; TAM is measured in adoption (installations, missions created), not revenue.

Q10: What evidence do we have that customers want this?

Evidence is structural and observational. No customer interviews have been conducted.

Identity gap:

- An analysis of 253 CLAUDE.md files found they are "primarily optimized for efficient code execution" and not for encoding who the user is [1]. A study of 2,303 context files across 1,925 repositories confirmed agents operate without identity context [2].
- SoulSpec emerged as a community standard for agent personas, adopted in 466 open-source projects [6]. This confirms demand for structured identity, but only for agents, not humans, and without delegation or cross-tool integration.
- Claude Managed Agents (April 2026) [13] uses "persona" for deployment configuration, not rich identity. Human developer identity is absent from the model.

Delegation gap:

- 93% of popular agent projects use unscoped API keys with no delegation controls [3]. 48.9% of organizations are entirely blind to machine-to-machine traffic [14].
- OWASP identifies Identity and Privilege Abuse as a top agentic AI risk: "agents need their own identities with task-scoped, time-bound permissions and clear auditability" [15].
- Academic research confirms 2–3 reflection turns achieve unanimous convergence in multi-agent decision-making [16], validating ethos's bounded-rounds-with-reflection design.
- NIST COSAiS (draft Q3 2026) targets least-privilege tool access, agent action containment, and chain-of-custody logging [17], exactly the controls ethos's workflow pillar implements.

Internal observation: Three independently developed tools (messaging, voice, email) each reinvented a fragment of identity before a shared layer existed. One live mission (ethos-db7, April 2026) went from bead claim to merged PR in 12 minutes 55 seconds (17% coding, 62% review pipeline), demonstrating that the mission primitive works in practice.

Validation plan: Interview 10 developers who use Claude Code with sub-agents or Agent Teams. The discriminating question: is the combined identity + delegation gap painful enough to install a tool that fills both?

Q11: Who are the competitors and why will we win?

- **SoulSpec / OpenClaw** [4, 6]: structured agent personas (SOUL.md, IDENTITY.md, STYLE.md). Agent-only; no human identity, no teams, no delegation, no MCP/CLI surface. 370,000+ GitHub stars via OpenClaw. The closest identity competitor. Ethos ingests SoulSpec files as an onboarding path: SoulSpec users import their existing personas and gain structural enforcement (typed roles, team graphs, mission contracts) without starting from scratch. Ethos differentiates on human-agent parity, team graphs, and the workflow pillar.
- **Mastra** [8]: typed Zod request schemas and pre-delegation hooks. 300,000+ weekly npm downloads. Closest to typed delegation. No write-set boundaries, no frozen evaluator, no append-only audit, no identity layer.
- **Claude Managed Agents** [13]: Anthropic's hosted agent infrastructure (April 2026). Stateful sessions, scoped permissions. "Persona" means deployment configuration, not rich identity. No human developer identity. Vendor-specific.
- **A2A v1.0** [5]: 150+ organizations, Signed Agent Cards for cryptographic identity. Agent discovery protocol, not a persona or delegation layer. No human identity model.
- **CLAUDE.md / AGENTS.md** [7]: per-project conventions, not per-person identity. Complementary.
- **CrewAI**: role-based agent orchestration. Natural-language delegation, no typed contracts, no persistent identity, no write-set or reflection.
- **Entire.io** [18, 19, 20]: Thomas Dohmke (ex-GitHub CEO), $60M seed at $300M valuation. Entire captures session fidelity (every token, tool call, and prompt) as retrospective checkpoints stored on a dedicated git branch (`entire/checkpoints/v1`). AI-generated summaries (intent, outcome, learnings, friction, open items) attach to commits after the session ends. The critical difference: Entire records what the AI did (retrospective forensics); ethos specifies what the AI should do before it acts (prospective contracts). Entire has no pre-delegation spec, no write-set boundaries, no typed mission lifecycle, no ADRs. Ethos has no full-transcript capture; that role is filled by Quarry (optional). The two are complementary: Entire answers "what did the AI do?" Ethos answers "what did we ask it to do, why, and did it stay in bounds?"
- **gstack** [21]: 23 SKILL.md prompt files by Garry Tan (YC CEO) simulating an engineering sprint team. Not a competitor but a validation: gstack proves developers want sprint discipline for agent teams. The content is high-quality workflow design with no structural enforcement: write-sets are suggestions, safety is conversational, personas reset each invocation. Ethos provides the enforcement layer that makes prompt-based workflows structural. Complementary: a gstack-inspired sprint team ships as a batteries-included ethos starter template.

Why ethos wins: No competitor combines human-agent identity parity, typed delegation contracts with file-level ownership, and a multi-surface integration pattern. SoulSpec has identity (agent-only). Mastra has delegation hooks (no identity). A2A has discovery (no personas, no delegation). Ethos is the only tool where the same runtime gives an agent its personality, constrains its file access in a mission, and lets a voice tool read its voice config, all from the same identity record.

Competitive response: If Anthropic ships native human identity in Claude Code, the identity pillar becomes less necessary as a standalone layer. The workflow pillar (missions, write-sets, audit trails) and the cross-tool integration pattern would retain independent value. Entire.io validates the governance market at a $300M valuation [18], confirming that intent capture and audit trails for agent-written code are a real category. Ethos's approach is prospective (contracts

before delegation) rather than retrospective (checkpoints after commits), and built-in rather than a separate product. The bet is that the agentic tool market remains fragmented and that identity + delegation + integration need to be one layer.

Q12: What is the customer acquisition strategy?

Five channels, ordered by expected contribution:

1. **Organic search and GitHub discovery.** Developers searching for "Claude Code identity," "agent delegation," or "AI team structure" find the repo. The MIT license and zero-dependency install remove friction. README, SEO-targeted docs, and the live example walkthrough convert visitors to installers.
2. **Punt Labs tool ecosystem.** Biff, Vox, and Beadle surface ethos during normal use: "install ethos for richer identity context" prompts appear when the tool detects ethos is absent. Each tool is a distribution channel for the shared layer.
3. **Community and content.** Blog posts, conference talks, and example repos demonstrating mission contracts and team structures. The 12m55s end-to-end demo is the core asset: a real bug fix from bead claim to merged PR, timed and reproducible.
4. **Portable export and ecosystem interoperability.** Lossy export (Feature 20) lets ethos users publish identities into two ecosystems: SoulSpec files (SOUL.md, IDENTITY.md, STYLE.md, AGENTS.md) for OpenClaw and compatible tools, and CLAUDE.md personal sections for any Claude Code session without ethos installed. A2A-compatible discovery lets ethos identities participate in cross-platform agent networks. Developers in those ecosystems discover ethos when they want human identity parity, team graphs, or delegation contracts that their current tools lack.
5. **Content compatibility.** Ethos imports structured workflows from popular prompt-based tools. gstack's 23 SKILL.md files [21] become 6 identities with enforced roles; SoulSpec personas map to ethos identities with team structure. Users of these tools discover ethos when they want structural enforcement behind their prompt-based workflows: "you already have the workflow discipline as prompts; ethos makes it structural." A gstack-inspired sprint team ships as a batteries-included starter template, so developers start with a working team structure on day one.

Assumption: Channel contribution is judgment-based. No acquisition data exists. The validation interviews will test whether "found it on GitHub" or "heard about it from a tool prompt" is the dominant path.

Q13: What is your next step to validate your vision?

Interview 10 developers who use Claude Code with sub-agents or Agent Teams. The discriminating question: **is the combined identity + delegation gap painful enough to install a tool that fills both?**

The interview protocol:

1. Do you configure personal preferences in CLAUDE.md? How many repos?
2. When you delegate to a sub-agent, how do you specify what files it can touch? What happens when two agents edit the same file?
3. **Demo:** Show (a) session where the agent greets the developer by name with a distinct personality, (b) a mission contract that bounds a delegation to specific files with 2 rounds max, (c) the audit trail after the mission closes.
4. After seeing this: would you install a tool that provides identity + delegation structure, knowing it takes 5 minutes and stores everything locally?

5. If Claude Code added native identity and delegation next month, would you still want a cross-tool layer?

Success threshold: 7 of 10 describe the combined gap as a daily friction or would install ethos after the demo. If most say "nice but I'd wait for Claude Code," the gap is real but the urgency is insufficient for a standalone tool.

Technical

Q14: What are the major technical risks?

Risk 1: Integration surface adoption. Three integration patterns (filesystem, CLI, MCP) require documentation and examples. Third-party adoption is unproven. Mitigation: three first-party tools (Biff, Vox, Beadle) demonstrate the patterns. Documentation ships with the binary.

Risk 2: Mission contract complexity. The workflow pillar has 7 primitives (contract, write-set, frozen evaluator, bounded rounds, verifier isolation, result artifacts, event log). New users may find the full lifecycle overwhelming. Mitigation: the mission is optional; identity works without it. The spectrum from tight implementation to open design means users can start simple. The live example (12m55s, docs/example/) demonstrates the full flow.

Risk 3: Schema evolution. Core identity fields are fixed; extensions live in separate files per tool. Mission contracts use strict unknown-field-rejecting decode. Adding fields requires careful versioning. Mitigation: the extension model handles per-tool data without core schema changes. The strict decoder prevents silent corruption.

Risk 4: SoulSpec competition. SoulSpec is gaining traction (466 OSS projects, OpenClaw adoption). If SoulSpec expands to include human identity and cross-tool integration, ethos's identity pillar faces direct competition. Mitigation: the workflow pillar and multi-surface integration are independent of the identity competition. Lossy export to SoulSpec and CLAUDE.md formats (Feature 20) turns this into coexistence rather than competition: ethos ingests SoulSpec files on the way in and exports them on the way out.

Q15: What dependencies exist on other teams or systems?

None. Ethos is a standalone Go binary with zero runtime dependencies. It does not depend on Biff, Vox, Beadle, or any external service. The reverse dependency is graduated: filesystem readers have no binary dependency; CLI and MCP callers need the binary installed. If ethos is absent, tools fall back to their own identity resolution.

Q16: What is the development status?

v4.9.0 shipped. 45 KLOC production Go, 65 KLOC tests, golangci-lint clean.

Shipped (v4.9.0): Identity, sessions, persona animation, teams, roles, anti-responsibilities, all 7 workflow primitives, safety constraints, audit logging (live write path plus sealed chunks; committed by default today, moving to machine-local under DES-079), ADRs, archetypes, import/export, team bundles with three-layer resolution, foundation and gstack bundles, the CEO/COO default team structure (human CEO, `claude` COO), `ethos setup` wizard, CLI-only install (`-no-plugin`), the `schema` subcommand (`ethos identity/role/team schema`), 13 pipeline templates, mission dispatch one-liner, automatic mission traceability, self-standing repos (`ethos vendor`, `resolution: repo-only`, ext `.local`), path and address redaction in the audit writer and path redaction in the delegation skeleton writer, the mission-lifecycle-cluster correctness fixes, PR/FAQ pipeline integration (LaTeX output, meeting personas, peer review).

In progress: Per-agent mission memory: quarry already supports agent-scoped `remember` and `find`. Remaining: ethos-side hooks for mission-close takeaways and mission-dispatch prior-work search (ethos-d17). Git tiers (DES-079, ethos-g3g3): the blame tier is tracked by default and the trace tier moves to machine-local storage with opt-in export. The ADR store moving into the repository (DES-079, ethos-yjh0). Redaction of paths, email addresses, and credentials in the mission-record and audit writers (DES-080, ethos-3e3f). All are decided, not shipped. Remaining work is integration-driven: ethos-side hooks for quarry agent memory, and third-party tool integrations. Customer validation interviews are the next adoption gate.

Q17: What is the scaling story?

Ethos is a local tool: no server, no database. Scaling means identities on one machine and concurrent sessions reading them. At 100 identities and 50 concurrent sessions, the bottleneck is filesystem I/O for flock contention, which Go handles efficiently. Mission records are per-repository files. No user will realistically hit these limits.

Q18: How does ethos capture intent and maintain audit trails?

Three layers, each independent:

Git (durable, shared): the blame tier. Commit trailers (`Mission:`, `Delegation:`), mission contracts, structured results, delegation records, per-mission corrections (`corrections.yaml`, a record of corrections to a closed mission's outcome, so a correction reaches every clone; decided, in progress), and the `missions.jsonl` trace log live in the repository. The contract captures intent before delegation: what was specified, what files are owned, what success looks like. The result captures outcome: what was delivered, what passed, what didn't. Together with `git blame`, these answer which mission, agent, and contract produced a line. They travel with the code in version control. Typed ADRs capture design decisions with a status lifecycle (proposed → settled → superseded), rejected alternatives, and links to the missions where decisions were made [22]. ADRs will be stored in the repository as `.punt-labs/ethos/adrs/<id>.yaml`, alongside contracts and results: decided under DES-079, in progress, not shipped (bead ethos-yjh0). Today they are stored globally in `~/.punt-labs/ethos/adrs` and are not tracked by git.

Local (operational): the trace tier. While a session runs, per-tool-call audit lines write to a machine-local, gitignored live file (so a repo with an active session keeps a clean `git status`). The pre-commit seal turns them into immutable, timestamp-named chunks. Delegation prompts, reflections, mission event logs, and sealed audit chunks form the trace tier, used for debugging and post-mortems. Today the delegation prompts, `reflections.yaml`, and the sealed event-log chunks are committed in the tracked mission tree, and the sealed audit chunks are committed in the sessions tree. So are the agent-written `artifacts/*.md` files, which agents write with their own file tools. Ethos never writes, exports, or redacts them.

In progress. This two-tier split is decided (DES-079, DES-080) but not yet implemented. Under DES-079 and the default `record: blame`, the trace tier will go to a machine-local archive, `.punt-labs/local/ethos/trace/`, and stay on the developer's machine. The live event log and live audit file are always machine-local. It will enter git only by choice: `ethos mission export <id> -full` will promote one mission's prompts, reflections, and event-log chunks through the redactor, and `record: full` in the repo config will write the trace-tier files ethos writes to their tracked paths, audit chunks included. The live files and agent-written artifacts are excluded. Audit chunks are per session, so `-full` does not export them and they reach git only under `record: full`. Under `record: blame`, `ethos enable` writes ignore rules for the tracked trace-tier paths, so `-full` output is ignored. The command prints a `git add -f <paths>` line to run, because

export does not stage its output. Under `record: full`, `ethos enable` omits those rules. Agent-written artifacts are ignored in both modes, and neither `-full` nor `record: full` touches them. Today the sealed audit chunks, the sealed event-log chunks, delegation prompts, reflections, and the agent-written artifacts are committed. The writers for `contract.yaml`, `results.yaml`, `reflections.yaml`, `missions.jsonl`, the audit sentinel, the legacy-audit migration, `ethos mission migrate`, and export do not redact yet, the event log is unredacted except for three events, and no credential pass exists (details in FAQ 5). Existing chunks stay in git history and are not rewritten.
Quarry (optional, semantic). When Quarry is installed, conversation transcripts are captured before context compaction and tagged with the active mission ID. Semantic search across sessions finds the reasoning behind decisions: "why did we choose conventions over enforcement?" retrieves the conversation where DES-041 was discussed. Quarry adds the unstructured "why" layer that structured artifacts cannot capture.
Together: git answers "what was decided," local logs answer "what happened in detail," Quarry answers "why was it decided." Each layer works independently. This is a lightweight built-in alternative to Entire.io's checkpoint-branch approach [19], with zero additional infrastructure for teams already using git. Entire captures the full transcript retrospectively; ethos captures intent prospectively in the contract and outcome structurally in the result. The two approaches are complementary rather than competing.

Business

Q19: What is the revenue model?

Free and open-source. The strategic value is adoption-driven: wider ethos adoption increases the value of every tool built on the identity and workflow layer. When a developer installs ethos once, messaging, voice, email, code review, and any future tool gain richer context without reimplementing identity or delegation.

Q20: What does the P&L look like at steady state?

Ethos has no revenue line. The P&L is a cost analysis.
Costs (annual): Engineering maintenance: 40–80 hours/year (bug fixes, Go upgrades, schema evolution). CI/CD: negligible (<$50/year). No infrastructure cost.
Strategic value (assumed): If any downstream tool reaches 200 users and ethos contributes a 10% retention improvement, the value exceeds maintenance cost. Testable once downstream tools have users.
Decision threshold: If after 12 months ethos has fewer than 100 installations and no measurable lift to downstream tool retention, fold identity and workflow into each consumer tool and deprecate the standalone binary.

Q21: What are the key metrics?

1. **Installations**: 500 in 6 months (GitHub release downloads).
2. **Missions created**: evidence that the workflow pillar is used, not just identity. Target: 100 missions in 6 months.
3. **Tool integrations**: 4 within 6 months (first-party + third-party).
4. **Third-party extensions**: 1 within 12 months.

Decision threshold: installations below 100 after 6 months = identity layer is not a pull product. Missions below 10 = workflow pillar is not valued. Either triggers reconsideration.

Reference class note: The 500-install and 100-mission targets are judgment-based. No direct reference class exists for a free, open-source identity-and-workflow tool in this product category. The decision thresholds (100 installs, 10 missions) are the actionable numbers: they determine whether ethos continues as a standalone product. The higher targets are aspirational indicators, not kill criteria.

Q22: What are we not building?

Two strategic exclusions deserve explanation (see also Feature 41 and Feature 38 in the Feature Appendix):

Runtime enforcement. Write-sets are conventions: declared in the mission contract, surfaced to the agent, and verified in review. Ethos does not enforce them at the filesystem level. This is deliberate. Agent runtimes (Claude Code, Codex CLI) do not expose a sandboxing API today. Building kernel-level containment would mean shipping a sandbox product, not an identity product; a different problem with different customers. If agent runtimes add scoped filesystem access in the future, ethos can emit the constraints for them to enforce. Until then, convention-plus-review is the only mechanism that works across all runtimes.

Cloud sync. Ethos has no server. The mission blame record and any repo-local or vendored identity graph content live in the repository and reach other machines only through git (global content in `~/.punt-labs/ethos/` does not, so a fresh clone has the complete graph only when the repository vendors it); session state stays on the developer's machine, and ADRs are global today and will be in the repository once DES-079 ships (FAQ 5). This blocks the most obvious enterprise adoption path (centralized team management, SSO-provisioned identities) but keeps the tool simple, private, and zero-dependency. The tradeoff is intentional: having no server means no authentication, no trust model, and no server infrastructure. A team service is under design in DES-081 and needs an amendment to this PR/FAQ.

Q23: Why now?

Five shifts converged:

1. **Agent coding went mainstream.** Claude Code is the most-loved AI coding tool (46%, $n \approx 906$) [10]. 84% of developers use or plan to use AI tools [23]. Multi-agent sessions (Agent Teams, sub-agents) created 3–10 agent scenarios where anonymous coordination breaks.
2. **The delegation gap became a security problem.** 93% of agent projects use unscoped credentials [3]. 48.9% of organizations are blind to machine-to-machine traffic [14]. NIST is drafting agent governance standards for 2027 [17]. Ethos ships the controls ahead of the standard.
3. **Agent identity became a real category.** SoulSpec launched as an open standard [4]. Claude Managed Agents uses "persona" in its model [13]. A2A v1.0 introduced Signed Agent Cards [5]. The vocabulary and demand exist, but no tool addresses humans, delegation, and cross-tool integration together.
4. **The community built workarounds.** SOUL.md, IDENTITY.md, USER.md appeared as informal conventions [24]. None standardized, none portable, none covering delegation. Ethos provides the structured, portable layer these workarounds approximate.
5. **The market is growing fast enough to support new infrastructure.** The AI agent market is $7.63B in 2025, growing at 45.8% CAGR to $50.31B by 2030 [25]. 79% of enterprises have adopted AI agents, but only 11% are in production; the adoption-production gap is where governance tools like ethos create value. Gartner projects 40% of enterprise apps will

embed task-specific AI agents by end of 2026 [26]. Separately, Gartner warns that over 40% of agentic AI projects risk cancellation by end of 2027 without adequate governance [27].

Risk Assessment

Risk	Rating	Assessment
Value	Medium	The identity gap is structurally confirmed (253 CLAUDE.md files, 466 SoulSpec projects, OWASP top 10). The delegation gap is confirmed (93% unscoped credentials, 48.9% blind to M2M traffic). The combined product (identity + workflow + integration in one layer) is untested with customers. SoulSpec validates identity demand but only for agents. Missions validated in practice (12m55s end-to-end) but not by external users. The "pull" signal is untested: do developers want a unified layer, or do they expect identity and delegation from separate tools?
Usability	Medium	Identity onboarding is simple (three commands, five minutes). The mission workflow introduces 7 primitives that have not been tested with external users. The live example demonstrates feasibility, not usability; it was operated by the builder.
Feasibility	Low	Shipped through v4.9.0. 45 KLOC production Go, 65 KLOC tests, golangci-lint clean. All 7 workflow primitives, 7 lifecycle hooks, audit logging (live plus sealed; the git-tier change in DES-079 is in progress), safety constraints, team bundles, setup wizard, 13 pipeline templates, 10 archetypes, and 2 embedded bundles are implemented and tested. Self-standing repos (`ethos vendor`, repo-only resolution) ship in v4.9.0. No unsolved technical problems.
Viability	Medium	Ethos is free infrastructure whose viability depends on adoption and downstream tool value. Maintenance cost is low (40–80 hrs/year). If SoulSpec captures agent identity and a dominant tool ships native delegation, both pillars face competition. The cross-tool integration pattern is the most defensible value: it is the only structural advantage that requires a shared layer rather than per-tool features.

Pre-Mortem

Three scenarios would cause ethos to fail:

1. **SoulSpec wins agent identity; a dominant tool ships delegation.** If SoulSpec becomes the standard for agent personas and Claude Code ships native mission contracts, ethos is squeezed from both sides. The cross-tool integration pattern survives only if the market stays fragmented. Mitigation: ethos ingests SoulSpec files and exports to both SoulSpec and CLAUDE.md formats, interoperating rather than competing.
2. **The combined product is too broad.** Identity + workflow + integration may be too much for one tool. Developers may want "just identity" or "just delegation" from focused tools. Mitigation: each pillar works independently. Identity loads without missions. Missions work without team structure. The pillars compound but don't require each other.
3. **The benefit is too abstract to motivate installation.** Developers may understand the gaps intellectually but not feel them strongly enough to install a tool, especially when Claude

Code adds native features quarterly. Validation interviews will test this. If 7 of 10 say "I'd wait for native," the standalone tool should be reconsidered.

Feature Appendix

Must Do (Shipped)

- **F1.** **Identity CRUD** — create, read, update, delete personas: same schema for humans and agents
- **F2.** **Persona animation** — SessionStart, PreCompact, and SubagentStart hooks inject personality, writing style, and talents automatically
- **F3.** **Teams and roles** — team graph with collaboration edges, role-based tool restrictions, anti-responsibility derivation from reports_to edges
- **F4.** **Agent file generation** — SessionStart generates .claude/agents/<handle>.md from identity, personality, role, and team data. Today this repository commits `.claude/agents/`. Decided in DES-079 and not shipped: committing the generated files in every repo so a fresh clone has its agents before the first session, deterministic generation, and an `ethos doctor` check that fails when the committed files differ from the generator's output
- **F5.** **Mission contracts** — typed delegation with leader, worker, evaluator, write-set, success criteria, and round budget
- **F6.** **Write-set admission** — segment-prefix conflict scan refuses overlapping file claims between open missions
- **F7.** **Frozen evaluator** — sha256 content hash pinned at mission launch; SubagentStart refuses verifier spawns if the hash drifts
- **F8.** **Bounded rounds with reflection** — hard round cap with mandatory structured reflection before advancing
- **F9.** **Verifier isolation** — verifier subagent receives only the contract and file allowlist, not the worker's scratch state
- **F10.** **Result artifacts and close gate** — typed results with verdict, confidence, files_changed, evidence; close refused without a valid result
- **F11.** **Append-only event log** — JSONL audit trail per mission; one corrupt line does not erase the tail
- **F12.** **SessionStart working context** — git branch, uncommitted changes, dirty file count injected at session start
- **F13.** **Role-based safety constraints** — tool + message pairs emitted in generated agent files
- **F14.** **Session audit logging** — PostToolUse hook appends one JSONL line per tool invocation to a per-session audit log
- **F15.** **Three integration surfaces** — filesystem (zero dependency), CLI (hooks/scripts), MCP server (12 tools)
- **F16.** **Extension model** — per-tool attributes under <handle>.ext/<tool>.yaml; any tool attaches config without schema changes
- **F17.** **Starter content** — 10 domain talents, 6 role archetypes, baseline-ops skill ship in the box
- **F18.** **Mission archetypes** — named contract conventions: implement (tight write-set, specific criteria, agent executes), design (broad write-set, open criteria, agent proposes and pushes back), review (read-only, findings with confidence), investigate (read-only, questions answered). Agency is expressed through contract specificity, not a mode switch
- **F19.** **Sprint team template** — a CEO/COO leadership pair over sprint specialists (architect, implementer, reviewer, QA, security) with writing styles, roles with safety constraints, and a team collaboration graph where specialists report to the COO
- **F20.** **Lossy export** — `ethos export -to soulspec|claude-md` compiles identities to portable formats. Structural data drops; the loss signals what ethos adds

- **F21.** **SoulSpec and CLAUDE.md ingestion** — `ethos import -from soulspec` reads SoulSpec files and creates a hardened ethos identity. LLM-assisted `ethos create` fine-tunes starter content from existing CLAUDE.md
- **F22.** **Typed ADRs** — Architecture Decision Records as ethos artifacts with schema validation, lifecycle status (proposed/settled/superseded), CLI, and MCP tool [22]
- **F23.** **Git-tracked mission export** — `ethos mission export <id> [-dir]` copies a mission's contract and final result to a directory you choose; by default it writes `.punt-labs/ethos/missions/<id>.contract.yaml` and `.punt-labs/ethos/missions/<id>.result.yaml`. Without `-dir`, export writes to `.punt-labs/ethos/missions/` under the repository root whatever the working directory, and fails outside a repository. An explicit `-dir` stays relative to the working directory. Under DES-079 (in progress) the blame tier (contract, results, delegation records, corrections, `missions.jsonl`, and that default export output) is tracked automatically, plain export keeps this behavior and will redact its output, and `-full` writes the mission's prompts, reflections, and event-log chunks to their tracked paths after redaction. It does not export audit chunks, which are per session and reach git only under `record: full`. Under the default `record: blame` those paths are covered by ignore rules, so `-full` prints a `git add -f <paths>` line to run (export does not stage its output); under `record: full` the rules are omitted. Agent-written artifacts are ignored in both modes, and ethos never exports or redacts them. Lightweight alternative to Entire.io's checkpoint branch [19]
- **F24.** **Integration guides** — filesystem, CLI, and MCP guides with working examples for third-party tools
- **F25.** **Team bundles** — config-driven switchable starter teams. Three-layer resolution: repo-local → active bundle → global. `ethos team activate/deactivate/available` (`migrate` shipped here too but removed post-ship, see DESIGN.md's DES-051 reversal note)
- **F26.** **Foundation bundle** — general-purpose team for any codebase: a CEO/COO leadership pair (human CEO, `claude` COO) over 4 specialists (architect, implementer, reviewer, security) that report to the COO. Language-agnostic. Ships embedded alongside gstack
- **F27.** **Setup wizard** — `ethos setup`: interactive wizard (name, handle, email, GitHub, working style) creates human + agent identities, writes repo config, activates bundle, generates agent files. `-solo`, `-bundle`, `-file`, `-json` flags
- **F28.** **Pipeline templates** — multi-stage mission workflows: 8 global pipelines (standard, quick, product, full, formal, docs, coverage, coe) plus 5 gstack-specific templates (13 total). `ethos mission pipeline instantiate`
- **F29.** **Mission dispatch** — one-liner creation from CLI flags: `-worker`, `-evaluator`, `-write-set`, `-criteria`
- **F30.** **Mission traceability** — close auto-appends a summary line to the tracked `missions.jsonl` trace log under `.punt-labs/ethos/` for commit-ready traceability
- **F31.** **PR/FAQ pipeline integration** — product pipeline's prfaq stage invokes the full Working Backwards workflow: research, structured discovery, meeting personas, peer review, LaTeX output
- **F32.** **Release externalization** — `/prfaq:externalize` generates a release-scoped press release from CHANGELOG entries
- **F33.** **Self-standing repos** — `ethos vendor` snapshots a resolvable identity set into the repo (following team-membership edges to a fixed point, planning by default); `resolution: repo-only` drops the global fallback so a missing reference is a hard error, not silent home-directory resolution; `<handle>.ext/<ns>.local.yaml` keeps secrets out of git. `ethos doctor` is the completeness gate

F34. **Sealed audit record** — per-tool-call audit lines write to a gitignored live file during a session and seal into immutable, timestamp-named chunks at pre-commit, which merge without conflict. Today the chunks are committed with the work. Decided and in progress (DES-079): under the default `record: blame` the seal writes to the machine-local archive `.punt-labs/local/ethos/trace/`, and audit chunks reach git only under `record: full`, because audit chunks are per session and `ethos mission export -full` does not export them. Event-log chunks also reach git through `-full`

Should Do

F35. **Per-agent mission memory (ethos hooks)** — quarry already supports agent-scoped `remember` and `find` with `agent_handle`. Remaining: ethos-side hooks: mission close prompts the agent for takeaways, mission dispatch searches the agent's collection for prior work

F36. **Quarry transcript tagging** — conversation transcripts captured before context compaction are tagged with the active mission ID. Semantic search retrieves the reasoning behind specific missions

F37. **Storage interface with an embedded local backend (ethos 2.0)** — runtime state moves behind one storage interface with an embedded local database per developer machine, replacing file locks and per-session sidecars with transactions. No daemon. The identity graph stays YAML files, repo-local, vendored, or global. Proposed in DES-081; design only, nothing implemented

Won't Do

F38. **Cloud sync or remote identity** — ethos has no server. Remote identity introduces trust, authentication, and privacy concerns that are out of scope for this layer. A team service (ethosd) is under design in DES-081; this item changes only through an amendment to this PR/FAQ.

F39. **Authentication or authorization** — ethos provides identity (who is this?), not access control (what can they do?). A different layer. A team service (ethosd) is under design in DES-081; this item changes only through an amendment to this PR/FAQ.

F40. **Consumer-specific fields** — the extension model exists for per-tool data. Ethos will never add fields for one consumer.

F41. **Runtime delegation enforcement** — write-set and safety constraints are surfaced to the agent and verified in review, not enforced at the filesystem level. Kernel-level sandboxing is a different product.

F42. **Telemetry or analytics** — no usage data collected. Metrics measured externally.

References

[1] Various. "On the Use of Agentic Coding Manifests". In: (2025). Analyzed 253 CLAUDE.md files from 242 repos. Manifests primarily optimized for code execution, not user identity. 8.7% include security content. URL: https://arxiv.org/abs/2509.14744.

[2] Various. "Agent READMEs". In: (2025). Analyzed 2,303 context files from 1,925 repos. Non-functional requirements rarely specified; agents operate without org-specific guardrails. URL: https://arxiv.org/html/2511.12884v1.

[3] Grantex. *State of AI Agent Security 2026*. Audit of 30 agent projects: 93% use unscoped API keys. No scope narrowing, depth limits, or cascade revocation. 2026. URL: https://grantex.dev/report/state-of-agent-security-2026 (visited on 04/09/2026).

[4] Peter Steinberger and SoulSpec Community. *SoulSpec — The Open Standard for AI Agent Personas*. Open standard for structured AI agent personas: SOUL.md, IDENTITY.md, AGENTS.md, STYLE.md. Agent-only — no human identity model. Native support in OpenClaw (370,000+ GitHub stars). 2026. URL: https://soulspec.org/ (visited on 04/09/2026).

[5] A2A Protocol Project / Linux Foundation. *A2A Protocol Surpasses 150 Organizations in First Year*. A2A v1.0 stable spec. Signed Agent Cards for cryptographic agent identity. No human participant identity model. 2026. URL: https://zexprwire.com/a2a-protocol-surpasses-150-organizations-lands-in-major-cloud-platforms-and-sees-enterprise-production-use-in-first-year/ (visited on 04/09/2026).

[6] Various. "Analyzing SOUL.md and Structured Persona Files in Open-Source AI Agent Repositories". In: (2026). MSR 2026. Large-scale empirical study of 466 OSS projects examining developer-defined AI agent personas. URL: https://arxiv.org/abs/2510.21413.

[7] pnote.eu. *AGENTS.md becomes the convention*. AGENTS.md and CLAUDE.md as cross-tool context files. Per-project, not per-person. No persistent identity model. 2025. URL: https://pnote.eu/notes/agents-md/ (visited on 02/17/2026).

[8] Mastra. *Supervisor Agents — Mastra Docs*. Typed Zod requestContextSchema and pre-delegation hook. 300,000+ weekly npm downloads. No write-set, frozen evaluator, or append-only audit. 2026. URL: https://mastra.ai/docs/agents/supervisor-agents (visited on 04/09/2026).

[9] Cisco. *2025 Data Privacy Benchmark Study*. 90% of organizations see local storage as inherently safer. 64% worry about sharing sensitive data with cloud GenAI tools. 2025. URL: https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2025/m04/cisco-2025-data-privacy-benchmark-study-privacy-landscape-grows-increasingly-complex-in-the-age-of-ai.html (visited on 03/19/2026).

[10] The Pragmatic Engineer. *State of AI Coding Tools: Developer Survey, February 2026*. n≈906 respondents (55% engineers, 34% leadership). Claude Code 46% most-loved (vs. Cursor 19%, Copilot 9%). 71% of agentic-tool users use Claude Code. 2026. URL: https://newsletter.pragmaticengineer.com/p/ai-tooling-2026 (visited on 04/09/2026).

[11] DemandSage. *Claude Statistics 2026*. Anthropic $14B annualized run-rate. Claude Code $2.5B run-rate. 18.9M monthly active web users. 300,000+ business customers. 2026. URL: https://www.demandsage.com/claude-ai-statistics/ (visited on 04/09/2026).

[12] PPCLand. *Claude Code reaches 115,000 developers*. 115,000 developers as of July 2025. $2.5B run-rate revenue in early 2026. Weekly active users doubled since January 2026. 2025. URL: https://ppcland.com/claude-code-statistics/ (visited on 03/19/2026).

[13] Anthropic. *Claude Managed Agents*. Launched April 8, 2026. Stateful sessions, scoped permissions. Agent = reusable versioned configuration. No human developer identity model. 2026. URL: https://claude.com/blog/claude-managed-agents (visited on 04/09/2026).

[14] Salt Security. *1H 2026 State of AI and API Security Report*. 48.9% of organizations blind to machine-to-machine traffic. 48% identify agentic AI as top attack vector. 2026. URL: https://salt.security/blog/the-era-of-agentic-security-is-here-key-findings-from-the-1h-2026-state-of-ai-and-api-security-report (visited on 04/09/2026).

[15] OWASP. *OWASP Top 10 Risks and Mitigations for Agentic AI Security*. ASI03: Identity and Privilege Abuse. Agents need their own identities with task-scoped, time-bound permissions and clear auditability. 2025. URL: https://genai.owasp.org/2025/12/09/owasp-genai-security-project-releases-top-10-risks-and-mitigations-for-agentic-ai-security/ (visited on 03/19/2026).

[16] Various. "Achieving Unanimous Consensus in Decision Making Using Multi-Agents". In: (2026). 2–3 reflection turns achieve unanimous convergence. Chain-of-thought prompts accelerate convergence. Validates bounded-rounds-with-reflection. URL: https://arxiv.org/html/2504.02128v1.

[17] NIST Computer Security Division. *SP 800-53 Control Overlays for Securing AI Systems (COSAiS)*. Targets least-privilege tool access, agent action containment, multi-agent trust boundaries, chain-of-custody logging. Draft Q3 2026. 2026. URL: https://csrc.nist.gov/projects/cosais (visited on 04/09/2026).

[18] TechCrunch. *Former GitHub CEO raises record $60M dev tool seed round at $300M valuation*. Entire.io by Thomas Dohmke. Targets human-agent coordination at the code governance layer. 2026. URL: https://techcrunch.com/2026/02/10/former-github-ceo-raises-record-60m-dev-tool-seed-round-at-300m-valuation/ (visited on 02/17/2026).

[19] Entire. *Core Concepts — Entire Documentation*. Checkpoint model: data on entire/checkpoints/v1 branch. Per-checkpoint: metadata.json, context.md (prompts), full.jsonl (transcript), attribution metadata. Append-only audit log. 2026. URL: https://docs.entire.io/core-concepts (visited on 04/09/2026).

[20] Thomas Dohmke and Entire. *Hello Entire World*. Founding post. Agent sessions are ephemeral — checkpoints make them durable. Three-component platform: git-compatible database, semantic reasoning, AI-native UI. 2026. URL: https://entire.io/blog/hello-entire-world (visited on 04/09/2026).

[21] Garry Tan. *gstack: 23 SKILL.md Files for Running a Sprint Team with Claude Code*. By Garry Tan (YC CEO). 23 SKILL.md prompt files simulating an engineering sprint team. No structural enforcement—write-sets are suggestions, personas reset each invocation. 2025. URL: https://github.com/garrytan/gstack (visited on 04/09/2026).

[22] Prompt Pals. *ADRs in the Age of AI*. ADRs as agent-readable governance artifacts. Identifies the gap: if ADRs live outside the repo boundary, agents are deprived of architectural intent. 2026. URL: https://prompt-pals.com/blog/adr-in-the-age-of-ai (visited on 04/09/2026).

[23] Stack Overflow. *2025 Developer Survey: AI*. 84% of developers use or plan to use AI tools. 23% use AI agents weekly. 2025. URL: https://survey.stackoverflow.co/2025/ai (visited on 03/19/2026).

[24] Blue Octopus Technology. *CLAUDE.md vs SOUL.md vs SKILL.md*. Community workarounds for identity: SOUL.md, IDENTITY.md, USER.md. None standardized or natively supported. 2025. URL: https://www.blueoctopustechnology.com/blog/claude-md-vs-soul-md-vs-skill-md (visited on 03/19/2026).

[25] DemandSage. *Latest AI Agents Statistics (2026)*. AI Agents market $7.63B in 2025 growing at 45.8% CAGR to $50.31B by 2030. 79% enterprise adoption; only 11% in production. 2026. URL: https://www.demandsage.com/ai-agents-statistics/ (visited on 04/09/2026).

[26] Gartner. *Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026*. 1,445% surge in multi-agent system inquiries Q1 2024 to Q2 2025. 40% of enterprise apps to embed AI agents by end of 2026, up from less than 5% in 2025. 2025. URL: https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025 (visited on 03/19/2026).

[27] Gartner. *Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027*. Over 40% of agentic AI projects risk cancellation by end of 2027 without governance frameworks and measurable ROI. 2025. URL: https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027 (visited on 05/11/2026).